Where LLM Knowledge Belongs: A Controlled Study of Semantic Injection Surfaces in SASRec-Style Sequential Recommendation

Jeremy Wizenfeld
jeremywizenfeld@gmail.com

Abstract

Large language models (LLMs) encode rich semantic information about items, but it remains unclear where that knowledge should enter a sequential recommender. Existing LLM-enhanced recommenders often combine semantic representations with additional architectural changes, making it difficult to determine whether improvements come from the LLM signal itself, the injection point, or the surrounding system design. In this work, we isolate the injection surface as the experimental variable under a fixed SASRec backbone and compare four representative ways of introducing frozen LLM-derived item representations: encoder input fusion, per-position encoder output distillation, sequence-level alignment, and item-table integration.

Within this controlled SASRec setting, our results suggest that the item embedding table is the most effective of the four surfaces studied. Unlike encoder-level or sequence-level methods, item-table integration shapes the shared item representations used for both sequence modeling and candidate scoring, allowing semantic structure to influence recommendation behavior without changing the deployed model. We further decompose item-table integration into two complementary mechanisms. A spectral initialization of the item table from frozen LLM embeddings provides a strong semantic warm start and primarily improves aggregate recommendation quality. A contrastive alignment loss applied directly to the item table provides additional pressure on sparse items under the sampled@100 protocol, helping preserve useful semantic structure for less frequently observed items that receive weaker collaborative supervision.

Together, these findings suggest that LLM knowledge is most effective when used to initialize and regularize the recommender's own item representations, rather than being attached as an auxiliary encoder. The LLM-derived embeddings are used only during training; at inference time, all auxiliary modules are discarded, leaving the deployed model architecturally identical to SASRec.

1 Introduction

Sequential recommendation predicts the next item a user will interact with based on their interaction history. Self-attentive models such as SASRec [2] and BERT4Rec [3] are now standard backbones, but they rely entirely on behavioral co-occurrence data: popular items receive abundant gradient signal while sparse items remain poorly calibrated regardless of encoder capacity.

Large language models offer a natural complement: through pretraining on large text corpora, LLM-derived item representations encode semantic structure that is available even for items with few interactions. Recent work has shown these priors improve sequential recommendation, particularly for sparse items [11, 12]. However, existing methods integrate semantic representations within larger systems involving adapters, dual-view encoders, retrieval modules, or LLM ranking distillation [10],

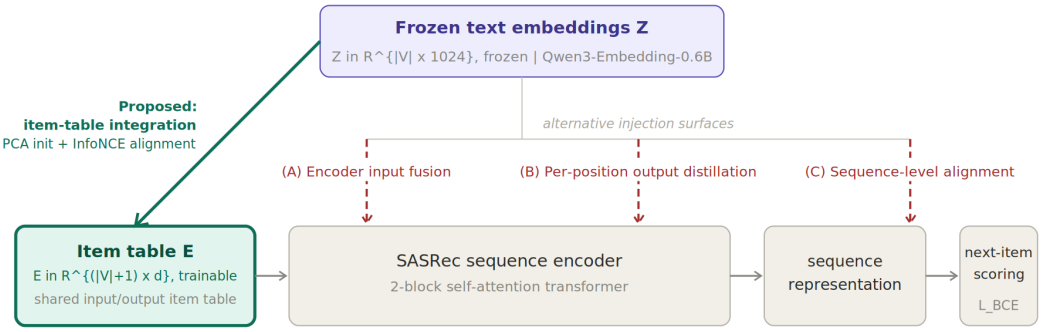

Figure 1: **The four semantic injection surfaces studied in this work, illustrated on a SASRec backbone.** Frozen text embeddings $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}| \times 1024}$ produced by Qwen3-Embedding-0.6B can enter the recommender at four distinct points. Dashed red arrows (A–C) inject into the sequence encoder: (A) encoder input fusion, (B) per-position output distillation, and (C) sequence-level alignment of the final user representation. The solid green arrow (ours) shapes the trainable item embedding table $\mathbf{E} \in \mathbb{R}^{(|\mathcal{V}|+1) \times d}$ directly, leaving the sequence encoder entirely untouched. Because $\mathbf{E}$ is shared between input encoding and next-item scoring, the proposed item-table integration propagates to candidate ranking at inference even though all auxiliary parameters are discarded after training.

entangling the choice of *where* the semantic signal enters with the choice of *how much* additional machinery surrounds it. As a result, the contribution of any specific injection point is difficult to isolate.

This leaves a basic design question unresolved: *where* should semantic signal enter a sequential recommender? We define a *semantic injection surface* as the point in the recommender pipeline at which frozen text-derived item representations are introduced, either as input features, per-position representation targets, sequence-level targets, or item-table constraints, and we treat the choice of surface as the experimental variable. As illustrated in Figure 1, four natural surfaces arise in a self-attentive recommender: encoder input, per-position encoder output, final sequence representation, and the item embedding table. We use Video Games as a development benchmark to compare all four surfaces under a fixed SASRec backbone, then test the winning surface against SASRec on three held-out datasets. Item-table integration is the strongest and most balanced (head + tail) surface on Video Games; the aggregate and sampled-tail gains transfer to Sports, Beauty, and Yelp.

Within item-table integration, a controlled decomposition shows that LLM-PCA initialization is the dominant lever for aggregate accuracy: it concentrates the warm-started item table on the principal directions of the frozen LLM space. Adding a symmetric InfoNCE alignment loss attached to the item table lifts sampled hit rate for long-tail items that the next-item BCE objective neglects; among the three weighting parameterizations we sweep on Video Games full-rank N@10 (Appendix C), a binary head/tail split is selected. These ingredients improve sampled-protocol performance across head, torso, and tail items on Video Games, and the aggregate and sampled-tail gains transfer to Sports, Beauty, and Yelp (1.3–1.9$\times$ sampled tail H@10), all with no inference-time overhead.

Contributions.

1. A controlled empirical study that isolates the injection surface as the experimental variable under a fixed SASRec backbone, comparing four representative surfaces and showing that item-table integration gives the best aggregate metrics and strongest sampled-tail metrics among the four surfaces studied.
2. Identification of popularity-dependent effects: initialization is the dominant driver of aggregate accuracy (concentrated on head items), and item-table InfoNCE alignment lifts sampled

hit rate for long-tail items that the next-item objective neglects. We do not claim recovery on the harder full-rank tail regime, where SASRec itself scores at or near zero.

3. A lightweight item-table framework combining LLM-PCA initialization with a symmetric InfoNCE alignment loss, adding no inference-time overhead relative to the SASRec backbone.

2 Related Work

Sequential recommendation. Early neural approaches modeled user dynamics with recurrent networks [1]; subsequent work adopted self-attention as the primary backbone, with SASRec [2] using causal self-attention for next-item prediction and BERT4Rec [3] adapting bidirectional transformers through masked item prediction. These models learn collaborative patterns effectively but rely entirely on behavioral data, leaving item representations poorly calibrated for sparse items.

Contrastive learning for sequential recommendation. Auxiliary self-supervised objectives improve representation robustness under sparsity via augmented sequence views [4, 6] or frequency-aware augmentation that avoids disproportionate harm to rare items [8]. Our frequency-weighted alignment shares the latter motivation but applies pressure at the item embedding table rather than the sequence encoder.

LLM-enhanced sequential recommendation. Several systems incorporate LLM-derived signals into sequential recommenders through different mechanisms. MoRec [9] studies ID- versus modality-based item representations as competing alternatives; DLLM2Rec [10] distills LLM ranking signals into a student recommender, altering the supervision signal rather than the injection point; LLMEmb [11] adapts LLM item embeddings through supervised contrastive fine-tuning and an adapter retained at serving time; and LLM-ESR [12], the closest system to ours in long-tail motivation, retains frozen semantic embeddings as a deployed dual-view component alongside the collaborative table. In contrast, we hold the SASRec backbone fixed and isolate the injection surface as the experimental variable, then propose an item-table method that uses frozen text embeddings only to initialize and regularize the trainable ID table; all auxiliary semantic modules are discarded after training.

3 Methodology

3.1 Problem Formulation and Backbone

Let $\mathcal{V}$ denote the item catalog with $|\mathcal{V}| = N$ items and, for each user u, let $S_u = (v_1^u, \ldots, v_{T_u}^u)$ be a chronologically ordered interaction sequence. We use SASRec [2] as a unified backbone across all experiments. An item embedding table $\mathbf{E} \in \mathbb{R}^{(N+1)\times d}$ (with index 0 reserved for padding) and a learned positional embedding produce input representations $\mathbf{h}_t^{(0)} = \mathbf{E}[v_t] + \mathbf{P}[t]$ that pass through B Transformer blocks with causal self-attention, yielding per-position output $\mathbf{h}_t^{(B)} \in \mathbb{R}^d$. We tie the input and output item tables, so $\mathbf{E}$ is used both for history lookup and candidate scoring via $s_{t,i} = \mathbf{h}_t^{(B)\top}\mathbf{E}[i]$ (raw dot product). The padding row $\mathbf{E}[0]$ is fixed at zero and excluded from PCA fitting, alignment batches, frequency counts, loss computation, and evaluation. Following the original paper, we use $d = 50$, $B = 2$ blocks, a single attention head, sequence length $L = 200$, and dropout 0.2.

Training uses next-item BCE: for each position t we pair the ground-truth next item with a uniformly sampled negative (rejected if it collides with any item in the user's training sequence, following standard SASRec practice; held-out validation and test items can in principle appear but the impact is negligible given catalog sizes of 10^4–10^5) and apply BCE against the position's score. For each item v we construct a short textual description from per-dataset metadata fields (Section 4.1; missing fields are omitted, not placeholdered) and encode it with a frozen Qwen3-Embedding-0.6B model [13], producing $\mathbf{Z}[v] \in \mathbb{R}^{1024}$. Embeddings are L2-normalized at extraction. $\mathbf{Z}$ is precomputed once per dataset and is not required at serving time.

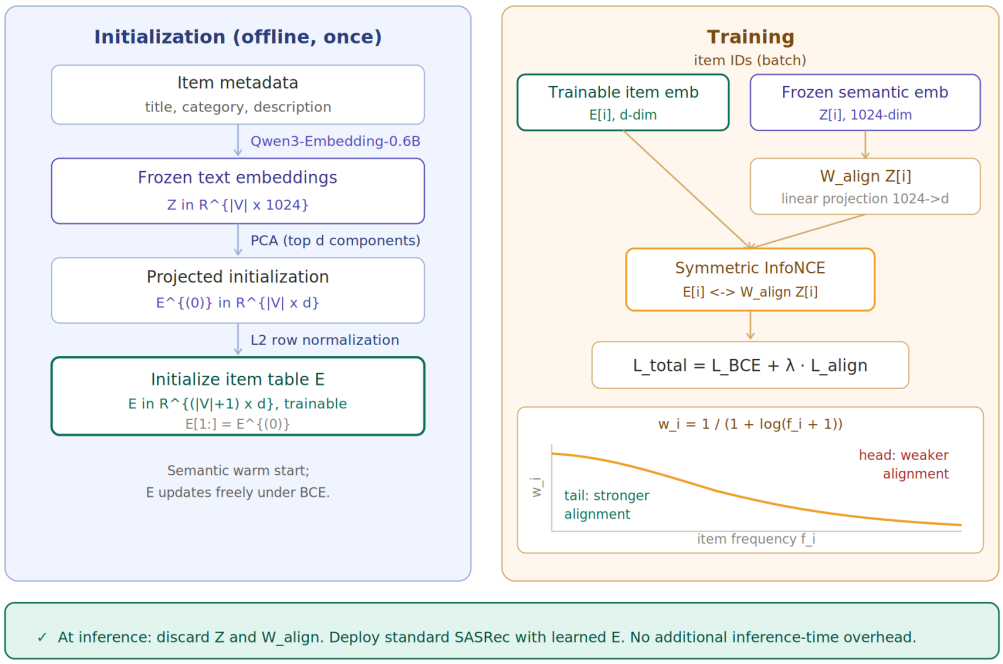

Figure 2: **Proposed item-table integration framework.** *Left, offline initialization:* item text is encoded by frozen Qwen3-Embedding-0.6B to produce $\mathbf{Z}[i] \in \mathbb{R}^{1024}$; a centered SVD compresses the catalog to $d = 50$ and seeds the trainable item table $\mathbf{E}[i]$ before training. *Right, training-time alignment:* for each batch of unique item ids, $\mathbf{E}[i]$ and $W_{\text{align}}\mathbf{Z}[i]$ are pulled together by a symmetric InfoNCE loss; per-item weights apply stronger pressure to low-frequency items (Equation 2). *Bottom, inference:* both $\mathbf{Z}$ and W_{align} are discarded and the deployed model is exactly the SASRec backbone.

3.2 Semantic Injection Surfaces

We instantiate four injection surfaces in the SASRec backbone (Figure 1). All share the same BCE objective.

(A) Encoder input fusion. $\mathbf{h}_t^{(0)} = \mathbf{E}[v_t] + W_{\text{in}}\,\mathbf{Z}[v_t] + \mathbf{P}[t]$, where $W_{\text{in}} \in \mathbb{R}^{d \times d_{\text{LLM}}}$ is learned; no auxiliary loss. Unlike the other surfaces, $\mathbf{Z}$ and W_{in} remain on the model at inference (every sequence step requires a lookup and a $d_{\text{LLM}} \times d$ matmul).

(B) Per-position output distillation. Inspired by knowledge distillation [19], a symmetric InfoNCE loss (temperature τ) aligns $\mathbf{h}_t^{(B)}$ with $W_{\text{out}}\,\mathbf{Z}[v_{t+1}^u]$ at every non-padding position, matching the next-item prediction target.

(C) Sequence-level alignment. A symmetric InfoNCE loss (temperature τ) aligns the final position output $\mathbf{h}_{L-1}^{(B)}$ (which under left-padding is the most recent real item's representation) with the mean-pooled semantic embedding $\frac{1}{T_u}\sum_t \mathbf{Z}[v_t^u]$ of the user's sequence.

Item-table integration (ours). The sequence encoder receives no auxiliary signal. $\mathbf{E}$ is shaped through (i) a one-time spectral initialization and (ii) a training-time symmetric InfoNCE alignment loss, both operating on the item table only (§3.3). Because $\mathbf{E}$ is tied between input encoding and output scoring, this shaping propagates to candidate ranking at inference even though all auxiliary parameters (W_{align}, $\mathbf{Z}$) are discarded after training. Surfaces (B), (C), and the proposed item-table integration add an auxiliary loss to BCE weighted by λ; (A) does not.

3.3 Item-Table Integration

LLM-PCA initialization. Let $\mathbf{Z} \in \mathbb{R}^{N \times d_{\text{LLM}}}$ denote the catalog matrix (padding excluded). After row-wise L2-normalization we center, decompose $\tilde{\mathbf{Z}} = U\Sigma V^\top$, and project to the top-d principal axes: $\mathbf{E}^{(0)} = \text{normalize}\big(\tilde{\mathbf{Z}} V_{:d}\big)$, re-L2-normalizing each row to unit length. We seed $\mathbf{E}[1{:}] \leftarrow \mathbf{E}^{(0)}$ and then let $\mathbf{E}$ update freely under BCE. The unit-norm rows differ in magnitude from the SASRec Xavier-uniform baseline. The PCA fit is offline and negligible relative to training time.

Frequency-weighted symmetric InfoNCE. The alignment objective uses the InfoNCE contrastive loss [16] applied symmetrically between the trainable item table and the projected frozen LLM table. Let $\mathcal{B}$ denote the unique non-padding item ids touched by a training batch, $|\mathcal{B}| = M$. For each $i \in \mathcal{B}$ we take $\mathbf{s}_i = \mathbf{E}[i]$ and $\mathbf{t}_i = W_{\text{align}}\,\mathbf{Z}[i]$, where $W_{\text{align}} \in \mathbb{R}^{d \times d_{\text{LLM}}}$ is a bias-free linear projector trained jointly with the model. With L2-normalized versions $\hat{\mathbf{s}}_i, \hat{\mathbf{t}}_i$ and temperature $\tau = 0.1$, we form $\mathbf{L}_{ij} = \hat{\mathbf{s}}_i^\top \hat{\mathbf{t}}_j / \tau$ and apply symmetric cross-entropy:

$$\mathcal{L}_{\text{InfoNCE}} = \tfrac{1}{2}\big(\text{CE}(\mathbf{L}, \mathbf{I}) + \text{CE}(\mathbf{L}^\top, \mathbf{I})\big). \tag{1}$$

To weight tail items more strongly we replace the unweighted mean inside each cross-entropy with a weighted mean using a non-increasing function of training-set frequency f_i,

$$w_i = g(f_i), \qquad \tilde{w}_i = M \cdot \frac{w_i}{\sum_{j \in \mathcal{B}} w_j}, \tag{2}$$

where $\tilde{w}_i$ normalizes to mean 1, preserving loss scale while increasing relative pressure on low-frequency items. We consider four parameterizations of g (logarithmic, square-root, linear, and a binary head/tail split) and select between them on validation full-rank N@10 (Appendix C); the selected variant is the binary split, $g(f_i) = 2$ for items in the bottom 20% of training-set frequency and $g(f_i) = 0.5$ otherwise.

3.4 Training and Inference

The total objective combines next-item BCE with the auxiliary alignment loss, $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \lambda \cdot \mathcal{L}^{(w)}_{\text{InfoNCE}}$, with λ selected on Video Games full-rank N@10 (§4). The alignment loss touches only rows of $\mathbf{E}$ that BCE is already updating, so the additional per-step cost is dominated by a single (M, M) matrix product. At inference time both $\mathbf{Z}$ and W_{align} are discarded: the deployed model contains only the SASRec parameters, adding *zero serving overhead*. Offline costs (one LLM forward pass per catalog item, one SVD, and per-step contrastive products during training) are incurred once per catalog version and must be re-run when the catalog changes.

4 Experimental Setup

4.1 Datasets

We evaluate on four sequential recommendation benchmarks: three Amazon Reviews 2023 categories from Hou et al. [14] (Video Games, Sports and Outdoors, and Beauty and Personal Care), and the Yelp Open Dataset [15]. For Amazon the LLM-encoder text input is `title + description`; for Yelp we reproduce the LLM-ESR [12] multi-field template verbatim. Full prompt templates and truncation rules are in Appendix A.

Preprocessing. For Amazon Reviews 2023 we download the `benchmark/5core/rating_only` subsets from Hou et al. [14], which are already iterative-bipartite 5-core fixed points: re-applying the filter drops zero users, items, and interactions on all three categories. For Yelp we apply iterative bipartite 5-core to the raw review log, alternating drops of users and items below five interactions until the table is stable. All four datasets are chronologically sorted per user and remapped to contiguous integer ids (id 0 reserved for padding).

Splits. We use a leave-one-out split [2, 3]: per-user, the last interaction is held out for test, the second-to-last for validation, and the remainder for training. After 5-core no users are dropped for being too short. Note that 5-core filtering means all evaluated items have at least five interactions; results speak to *sparse* items within this set, not true cold-start.

Table 1: Dataset statistics after 5-core preprocessing. The #Interactions column counts the full per-user sequences before the leave-one-out split removes two interactions per user.

Dataset	#Users	#Items	#Interactions	Avg. seq. len
Video Games	94,762	25,612	814,586	8.6
Sports and Outdoors	409,772	156,235	3,472,020	8.5
Beauty and Personal Care	729,576	207,649	6,624,441	9.1
Yelp	277,631	112,394	4,250,483	15.3

Table 2: Popularity bucket breakdown on the training split. Each bucket holds 20% / 60% / 20% of items by training-set frequency (i.e. after the leave-one-out split removes the val and test interactions). *Avg.* is the average number of training interactions per item in that bucket; *Share* is the bucket's percentage of total training interactions.

	Head		Torso		Tail	
Dataset	Avg.	Share	Avg.	Share	Avg.	Share
Video Games	87.1	71.1%	10.6	26.0%	3.5	2.8%
Sports and Outdoors	55.2	64.9%	8.8	31.2%	3.3	3.9%
Beauty and Personal Care	87.8	70.5%	11.0	26.4%	3.8	3.1%
Yelp	116.6	70.9%	14.5	26.5%	4.4	2.7%

Popularity strata. For the popularity-stratified evaluation in Section 5 we partition items by *training-set* frequency into head (top 20%), torso (middle 60%), and tail (bottom 20%) buckets. Cuts are computed on the training split only, so val/test leakage is avoided. All four datasets are strongly long-tailed: head 20% of items receive 65–71% of training interactions while tail 20% receive only 2.7–3.9%. Table 1 reports per-dataset statistics and Table 2 the per-bucket breakdown.

4.2 Baselines and Evaluation Protocol

Baselines. Our baseline is **SASRec** [2] trained with the exact backbone and recipe used by every variant (Section 4.3). All reported numbers come from our pipeline; we do not transcribe numbers from prior work because our preprocessing, sampled-eval seed pool, and full-rank protocol are end-to-end controlled, and direct comparison would conflate method differences with protocol differences. Surfaces (A)–(C) in Table 3 represent the main architectural locations at which prior LLM-enhanced systems introduce semantic representations, re-implemented here under our shared backbone.

Metrics. We report H@k and N@k for $k \in \{5, 10, 20\}$ under two ranking protocols. **Sampled@100** follows the standard SASRec-style sampled-negative evaluation [2]: each held-out target is ranked against 100 uniform-random negatives drawn once per user with a fixed seed and cached to disk, with the user's full sequence (train+val+test items) rejected from the negative pool. Sampled metrics are paired, so every variant ranks the same target against the same 100 negatives and variance reflects model differences rather than negative-pool differences. **Full-rank**: each target is ranked against all N catalog items with the user's training items and held-out validation item masked from the candidate set (the test target itself remains rankable). Scores are computed in mini-batches of users to bound memory on catalogs with $N > 10^5$. Full-rank metrics are much smaller in absolute magnitude than sampled metrics on large catalogs. We report both.

Validation and early stopping. Validation N@10 (sampled) is computed every epoch; training stops when validation has not improved by more than 10^{-3} for 10 consecutive epochs. The same criterion is applied across all datasets and all variants.

4.3 Hyperparameters and Hyperparameter Search

SASRec backbone (shared by all variants). We use the canonical SASRec [2] configuration: $d = 50$, two transformer blocks, single attention head, $L = 200$, dropout 0.2; item embeddings are tied between input and output. Optimization: Adam [18], lr 10^{-3}, batch 512, up to 200 epochs with early stopping (Section 4.2).

LLM encoder. Qwen3-Embedding-0.6B [13], frozen, producing 1024-dimensional vectors. Encoder weights are never updated; the LLM is not invoked at inference.

Item-table integration (proposed). The PCA initialization keeps the top $d = 50$ singular components of the L2-normalized catalog matrix and re-normalizes each row to unit length. PCA is chosen over a learned projection because it is parameter-free and directly targets the principal directions of the frozen LLM space. The alignment projector is a single bias-free linear map $\mathbf{W}_{\text{align}} \in \mathbb{R}^{d \times d_{\text{LLM}}}$ with input dropout 0.1. Symmetric InfoNCE makes the alignment bidirectional, pulling the collaborative table toward the LLM geometry and the projection toward the collaborative geometry, without requiring a teacher-student asymmetry. The contrastive temperature is fixed at $\tau = 0.1$ throughout. The alignment weight λ and the frequency-weighting function g are selected on Video Games full-rank N@10 over the grid $\lambda \in \{0.01, 0.05, 0.1, 0.5, 1.0\} \times g \in \{\text{log, sqrt, linear, binary}\}$; the selected configuration ($\lambda = 0.5$, $g =$ binary) is used across all four datasets. The binary split ($2\times$ for tail, $0.5\times$ otherwise) is interpretable and minimal; Appendix C confirms it is competitive with smoother alternatives. We use full-rank rather than sampled@100 as the model-selection criterion to avoid overfitting to the fixed 100-negative pool. To keep alignment mini-batches tractable, batches with more than $4{,}096$ unique item ids subsample to $4{,}096$; remaining ids backpropagate only through BCE. Per-batch weight normalization follows Equation 2.

Other surfaces. Surfaces (A), (B), and (C) use the same backbone and reuse the same $\tau = 0.1$ where applicable. Each surface's auxiliary loss weight is searched independently on Video Games N@10 from the same grid.

4.4 Reproducibility

All variants are trained with three random seeds $\{7, 18, 42\}$; we report mean $\pm$ std. The sampled-eval negative pool and LLM item embeddings are generated once per dataset and reused across all variants and seeds. Code, configs, and preprocessing scripts are available upon request.

5 Results and Analysis

Where should LLM knowledge enter? Table 3 compares the four injection surfaces under the shared SASRec backbone on Video Games. Item-table integration (ours) gives the best aggregate metrics on both sampled and full-rank protocols and the best tail-item metrics, while adding *zero* parameters at inference. Per-position distillation (B) is a close competitor: it ties on the second-best aggregate full-rank H@10 and slightly outperforms our method on full-rank head-item H@10 / N@10, but trails substantially on tail items. Input fusion (A) lifts vanilla SASRec slightly but keeps the LLM lookup and projector at serving cost. Sequence-level alignment (C) is the weakest: aligning one user-summary vector against a mean-pooled semantic embedding does not propagate strongly to per-item scores. The injection surface materially affects performance even under a fixed backbone, LLM encoder, and tuning grid; among the four, only item-table integration combines competitive aggregate accuracy with a clear tail advantage.

Cross-dataset generalization. Table 4 tests whether the winning surface generalises beyond the development benchmark. Item-table integration improves over SASRec on every sampled-protocol cell across all three datasets, and on Sports and Beauty it wins every full-rank cell as well, a clean sweep across two short-sequence Amazon catalogs. Yelp shows the same sampled-protocol pattern (including a $1.3\times$ sampled tail lift), though full-rank metrics are noisier given that SASRec already scores near zero on Yelp tail items.

Inside item-table integration. Table 5 walks the $\langle$init, alignment, weighting$\rangle$ space on Video Games under sampled@100. The overall and head H@10 columns show that *initialization is the dominant lever*: LLM-PCA init lifts sampled overall H@10 from 0.7194 to 0.7522 and head H@10 from 0.8719 to 0.8872, and no subsequent combination of alignment or weighting moves either metric by more than ± 0.004. The torso and tail columns tell the complementary story: init alone lifts sampled tail from 0.2263 to 0.2848 and torso from 0.5188 to 0.5840, but combining it with frequency-weighted InfoNCE alignment goes furthest, reaching sampled tail H@10 of 0.3195 and torso H@10 of 0.5894 (the best of any configuration in both columns). The two ingredients act on

Table 3: Cross-surface comparison on Video Games. SMP: sampled@100; FULL: full-rank (N=25,612). **Bold**: best per (column, protocol); underline: second best. Mean ± std over seeds $\{7, 18, 42\}$.

		Overall		Head item		Tail item	
Method	Eval	H@10	N@10	H@10	N@10	H@10	N@10
SASRec	SMP	0.7194±.0021	0.4954±.0014	0.8719±.0004	0.6349±.0009	0.2263±.0052	0.1087±.0016
	FULL	0.0574±.0013	0.0300±.0006	0.0862±.0019	0.0452±.0012	0.0004±.0003	0.0002±.0002
Input fusion (A)	SMP	0.7324±.0016	0.5069±.0020	0.8750±.0016	0.6420±.0031	0.2539±.0070	0.1190±.0039
	FULL	0.0595±.0018	0.0306±.0010	0.0892±.0031	0.0461±.0018	0.0004±.0001	0.0002±.0001
Per-position InfoNCE (B)	SMP	0.7431±.0023	0.5255±.0016	0.8796±.0039	**0.6695**±.0043	0.2643±.0133	0.1174±.0089
	FULL	0.0764±.0016	0.0405±.0009	**0.1171**±.0022	**0.0623**±.0013	0.0001±.0001	0.0000±.0000
Sequence-level InfoNCE (C)	SMP	0.7183±.0013	0.4938±.0004	0.8717±.0010	0.6346±.0009	0.2232±.0019	0.1063±.0028
	FULL	0.0575±.0002	0.0304±.0000	0.0869±.0004	0.0462±.0001	0.0003±.0001	0.0001±.0001
Item-table (ours)	SMP	**0.7539**±.0009	**0.5339**±.0005	**0.8834**±.0015	0.6692±.0021	**0.3195**±.0079	**0.1512**±.0030
	FULL	**0.0775**±.0005	**0.0412**±.0004	0.1160±.0009	0.0620±.0006	**0.0022**±.0003	**0.0010**±.0001

Table 4: Cross-dataset generalization beyond Video Games: SASRec vs. item-table integration on three additional benchmarks. Reported under both SMP = sampled@100 and FULL = full-rank ranking. **Bold** marks the within-dataset best per (column, protocol). Mean ± std over seeds $\{7, 18, 42\}$.

			Overall		Head item		Tail item	
Dataset	Method	Eval	H@10	N@10	H@10	N@10	H@10	N@10
Sports	SASRec	SMP	0.6290±.0021	0.4041±.0020	0.8146±.0021	0.5569±.0024	0.1775±.0070	0.0795±.0063
		FULL	0.0101±.0006	0.0051±.0004	0.0168±.0011	0.0085±.0007	0.0000±.0000	0.0000±.0000
	Item-table (ours)	SMP	**0.6862**±.0008	**0.4634**±.0003	**0.8474**±.0016	**0.6170**±.0024	**0.2947**±.0066	**0.1368**±.0039
		FULL	**0.0193**±.0005	**0.0100**±.0003	**0.0319**±.0009	**0.0166**±.0005	**0.0003**±.0000	**0.0001**±.0000
Beauty	SASRec	SMP	0.6780±.0075	0.4560±.0065	0.8412±.0069	0.5893±.0077	0.1014±.0050	0.0437±.0018
		FULL	0.0136±.0009	0.0070±.0006	0.0192±.0013	0.0099±.0009	0.0000±.0000	0.0000±.0000
	Item-table (ours)	SMP	**0.7345**±.0016	**0.5151**±.0012	**0.8799**±.0008	**0.6474**±.0008	**0.1952**±.0025	**0.0909**±.0014
		FULL	**0.0216**±.0002	**0.0114**±.0002	**0.0304**±.0002	**0.0160**±.0002	**0.0002**±.0001	**0.0001**±.0000
Yelp	SASRec	SMP	0.9096±.0007	0.7026±.0007	0.9656±.0003	0.8027±.0006	0.6132±.0013	0.3328±.0013
		FULL	**0.0432**±.0005	**0.0239**±.0004	0.0577±.0008	0.0307±.0007	**0.0113**±.0009	**0.0081**±.0006
	Item-table (ours)	SMP	**0.9344**±.0011	**0.7147**±.0032	**0.9674**±.0005	**0.8104**±.0012	**0.7828**±.0064	**0.4075**±.0084
		FULL	0.0432±.0010	0.0223±.0006	**0.0642**±.0011	**0.0329**±.0006	0.0027±.0007	0.0016±.0005

orthogonal axes: init dominates head accuracy, frequency-weighted alignment dominates the sampled torso and tail lift. Figure 3 visualizes the head/torso/tail breakdown.

Table 5: Item-table integration decomposition on Video Games (Sampled@100). "Init" is Xavier (vanilla) vs LLM-PCA; "Align" is whether the symmetric InfoNCE alignment loss is attached to the item table; "Weight" is the alignment weighting scheme. Overall H@10 and head H@10 are mostly determined by initialization; torso and tail metrics are where alignment, and specifically frequency weighting on top of LLM-PCA, makes the difference. Mean ± std over seeds $\{7, 18, 42\}$.

			Sampled@100			
Init	Align	Weight	Overall H@10	Head H@10	Torso H@10	Tail H@10
Xavier	—	—	0.7194±.0021	0.8719±.0004	0.5188±.0059	0.2263±.0052
LLM-PCA	—	—	0.7522±.0009	0.8872±.0015	0.5840±.0037	0.2848±.0060
Xavier	InfoNCE	uniform	**0.7572**±.0016	**0.8909**±.0006	0.5854±.0038	0.3114±.0031
LLM-PCA	InfoNCE	uniform	0.7556±.0009	0.8900±.0018	0.5816±.0019	0.3116±.0017
Xavier	InfoNCE	freq	0.7446±.0012	0.8768±.0013	0.5711±.0018	0.3153±.0060
LLM-PCA	**InfoNCE**	**freq (ours)**	0.7539±.0009	0.8836±.0016	**0.5894**±.0028	**0.3195**±.0064

Popularity stratification. Under sampled@100, where absolute numbers are interpretable, item-table integration lifts Video Games tail H@10 from 0.2263 (SASRec) to 0.3195, a $1.4\times$ relative improvement on top of an already non-trivial baseline; head items move slightly from 0.8719 to 0.8836. Figure 3 visualizes the sampled@100 head/torso/tail breakdown across decomposition configurations. LLM-PCA init lifts head and torso items; frequency-weighted alignment without LLM-PCA init pushes the tail further but at a slight head cost. Item-table integration achieves the best sampled tail (0.3195) while maintaining stronger head performance than any other configuration that reaches comparable tail lift.

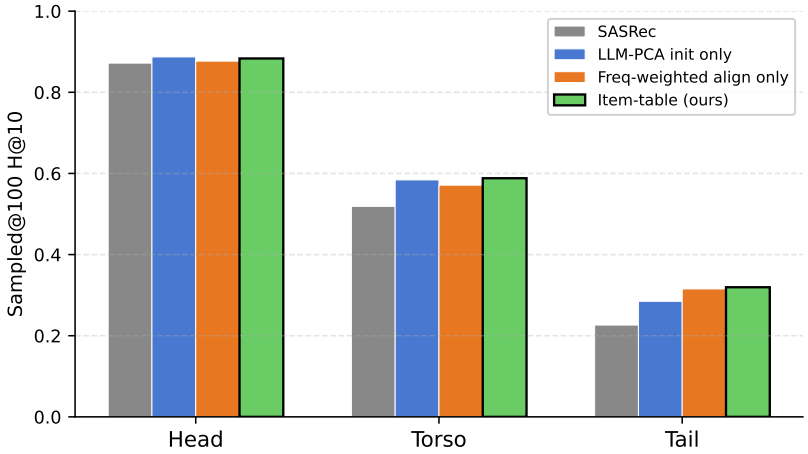

Figure 3: Sampled@100 H@10 on Video Games stratified by item popularity (head 20% / torso 60% / tail 20% by training frequency). LLM-PCA initialization lifts head and torso items; frequency-weighted alignment alone lifts the tail further but slightly reduces head; item-table integration achieves the highest tail while maintaining strong head performance.

Robustness. We additionally swept $\lambda \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$ and the weighting function $g \in \{\text{log, sqrt, linear, binary}\}$. All four weight functions beat vanilla SASRec on Video Games full-rank H@10; the three sub-logarithmic variants (binary, linear, sqrt) further beat the strongest non-item-table surface (B). Detailed sweep tables are in Appendix C and D. Marginal full-rank differences between surfaces (e.g. 0.0775 vs. 0.0764) are close to seed variance and should be interpreted cautiously.

6 Conclusion and Limitations

We isolated the *semantic injection surface* as the experimental variable for incorporating frozen LLM embeddings into a sequential recommender. Under a shared SASRec backbone, item-table integration gives the best aggregate and sampled-tail metrics among the four surfaces studied, with zero inference overhead. Item-table integration is uniquely effective because $\mathbf{E}$ is shared between input encoding and candidate scoring, so shaping it propagates semantic structure to ranking without any retained auxiliary parameters.

The mechanism decomposition shows that LLM-PCA initialization is the dominant lever for aggregate accuracy while also providing a meaningful sampled-tail lift; frequency-weighted InfoNCE alignment pushes the tail further to the best result of any configuration, with the combination preserving strong aggregate performance. These findings transfer across Sports, Beauty, and Yelp; we do not claim recovery in the full-rank tail regime, where SASRec itself scores near zero on three of four datasets.

Limitations. (i) Results are specific to SASRec (d=50); other backbones or larger d may reorder the surfaces. (ii) All results use Qwen3-Embedding-0.6B; the ranking may shift with encoders of different capacity, as each surface's benefit depends on encoder–collaborative geometry alignment. (iii) The method requires item text and must re-run embeddings and SVD when the catalog changes.

References

[1] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk. Session-based recommendations with recurrent neural networks. *ICLR*, 2016.

[2] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. *ICDM*, 2018.

[3] F. Sun et al. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. *CIKM*, 2019.

[4] X. Xie et al. Contrastive learning for sequential recommendation. *ICDE*, 2022.

[5] Z. Liu et al. Contrastive self-supervised sequential recommendation with robust augmentation. *arXiv*, 2021.

[6] R. Qiu et al. Contrastive learning for representation degeneration problem in sequential recommendation. *WSDM*, 2022.

[7] Z. Wang et al. Relative contrastive learning for sequential recommendation with similarity-based positive pair selection. arXiv:2504.19178, 2025.

[8] Z. Wang and W. Zhang. Frequency-aware adaptive contrastive learning for sequential recommendation. arXiv:2601.17057, 2026.

[9] Y. Yuan, G. Zhang, X. Liu, J. Wu, Y. Wang, M. Li, and J. Shi. Where to go next for recommender systems? ID- vs. modality-based recommender models revisited. *SIGIR*, 2023.

[10] Y. Cui, F. Liu, P. Wang, B. Wang, H. Tang, Y. Wan, J. Wang, and J. Chen. Distillation matters: Empowering sequential recommenders to match the performance of large language models. arXiv:2405.00338, 2024.

[11] Q. Liu, X. Wu, Y. Wang, Z. Zhang, F. Tian, Y. Zheng, and X. Zhao. LLMEmb: Large language model can be a good embedding generator for sequential recommendation. *AAAI*, 2025. arXiv:2409.19925.

[12] Q. Liu, X. Wu, Y. Wang, Z. Zhang, F. Tian, Y. Zheng, and X. Zhao. LLM-ESR: Large language models enhancement for long-tailed sequential recommendation. *NeurIPS*, 2024. arXiv:2405.20646.

[13] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. arXiv:2506.05176, 2025.

[14] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley. Bridging language and items for retrieval and recommendation. arXiv:2403.03952, 2024.

[15] Yelp Inc. Yelp Open Dataset. `https://www.yelp.com/dataset`, 2024.

[16] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018.

[17] S. Rajput, N. Mehta, A. Singh, R. Hulikal Keshavan, T. Vu, L. Heldt, L. Hong, Y. Tay, V. Q. Tran, J. Samost, M. Kula, E. H. Chi, and M. Sathiamoorthy. Recommender systems with generative retrieval. *NeurIPS*, 2023.

[18] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015. arXiv:1412.6980.

[19] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *NeurIPS Deep Learning Workshop*, 2015. arXiv:1503.02531.

A Prompt templates

Each item is encoded once with frozen Qwen3-Embedding-0.6B and the resulting 1024-dimensional vector is cached. Prompt templates are short, contain only fields available in the published dataset metadata, and were not tuned on validation; we expect prompt sensitivity to be a useful direction for future work.

Amazon Reviews 2023. Each item is formatted as `<title>` — `<description>`. The description is truncated at the last whitespace before 600 characters ($\approx$ 150 tokens) so a 10-item sequence stays under Qwen3's 32K-token context. Items with empty description fall back to `<title>`; items with empty title fall back to the literal string `Untitled`. Example:

```
Street Fighter 30th Anniversary Collection - Nintendo Switch
Standard Edition -- Celebrate the 30th Anniversary of the iconic
Street Fighter franchise with the ultimate tribute to its arcade
legacy in the Street Fighter 30th Anniversary Collection.
```

Yelp. We reproduce the LLM-ESR [12] template verbatim. Each business is formatted as:

```
The point of interest has following attributes:
 name is {name}; category is {category}; type is {type}; open
status is {is_open}; review count is {review_count}; city is {city};
average score is {stars}.
```

This template is chosen to make our Yelp prompt format directly comparable to prior LLM-for-SeqRec work on the same dataset. The fields are all native columns of `yelp_academic_dataset_business.json`.

Design rationale. We deliberately avoid LLM-prompt engineering: prompts are mechanical concatenations of available metadata. This isolates the contribution of injection-surface choice and avoids attributing gains to prompt tuning. A more elaborate prompting strategy (chain-of-thought attribute extraction, retrieval-augmented descriptions) could plausibly improve all four injection surfaces uniformly; we leave that to future work.

B Full @{5,10,20} metrics on Video Games

Tables 6 and 7 report H@k and N@k for $k \in \{5, 10, 20\}$ on Video Games for SASRec and the three LLM-injection surfaces of Section 5. The relative ordering is stable across k under both protocols; the @10 metrics used in the main paper are representative.

Table 6: Sampled@100 metrics at $k \in \{5, 10, 20\}$ on Video Games. Mean $\pm$ std over seeds $\{7, 18, 42\}$.

	H@k			N@k		
Method	@5	@10	@20	@5	@10	@20
SASRec	0.5943$\pm$.0003	0.7194$\pm$.0021	0.8273$\pm$.0013	0.4544$\pm$.0008	0.4954$\pm$.0014	0.5222$\pm$.0003
Input fusion (A)	0.6098$\pm$.0013	0.7324$\pm$.0016	0.8395$\pm$.0018	0.4672$\pm$.0021	0.5069$\pm$.0020	0.5341$\pm$.0018
Per-position InfoNCE (B)	<u>0.6245</u>$\pm$.0020	<u>0.7431</u>$\pm$.0023	<u>0.8461</u>$\pm$.0021	<u>0.4871</u>$\pm$.0017	<u>0.5255</u>$\pm$.0016	<u>0.5517</u>$\pm$.0014
Seq-level InfoNCE (C)	0.5947$\pm$.0013	0.7183$\pm$.0013	0.8287$\pm$.0012	0.4537$\pm$.0008	0.4938$\pm$.0004	0.5218$\pm$.0003
Item-table (ours)	**0.6356**$\pm$.0008	**0.7539**$\pm$.0009	**0.8572**$\pm$.0005	**0.4955**$\pm$.0006	**0.5339**$\pm$.0005	**0.5601**$\pm$.0004

C Weight-function sweep

Table 8 reports three alternative parameterizations of the frequency-weighting function g evaluated on Video Games at $\lambda = 0.1$. We report sampled@100 and full-rank H@10 / N@10 plus the popularity-stratified H@10 breakdown. The binary head/tail split is the strongest on full-rank metrics and is therefore used as the headline weighting in the main paper.

Table 7: Full-rank metrics at $k \in \{5, 10, 20\}$ on Video Games. Mean $\pm$ std over seeds $\{7, 18, 42\}$.

	H@k			N@k		
Method	@5	@10	@20	@5	@10	@20
SASRec	0.0353$\pm$.0005	0.0574$\pm$.0013	0.0902$\pm$.0013	0.0228$\pm$.0005	0.0300$\pm$.0006	0.0382$\pm$.0007
Input fusion (A)	0.0362$\pm$.0011	0.0595$\pm$.0018	0.0938$\pm$.0021	0.0231$\pm$.0008	0.0306$\pm$.0010	0.0392$\pm$.0011
Per-position InfoNCE (B)	0.0483$\pm$.0014	0.0764$\pm$.0016	0.1165$\pm$.0012	0.0314$\pm$.0008	0.0405$\pm$.0009	0.0505$\pm$.0008
Seq-level InfoNCE (C)	0.0359$\pm$.0002	0.0575$\pm$.0002	0.0905$\pm$.0004	0.0235$\pm$.0001	0.0304$\pm$.0000	0.0387$\pm$.0001
Item-table (ours)	**0.0489**$\pm$.0006	**0.0775**$\pm$.0005	**0.1176**$\pm$.0004	**0.0320**$\pm$.0004	**0.0412**$\pm$.0004	**0.0512**$\pm$.0003

Table 8: Weight-function ablation on Video Games at $\lambda = 0.1$, 3 seeds. *Binary* (head $0.5\times$, tail $2.0\times$) is the selected variant.

	Sampled@100		Full-rank		Full-rank H@10 by bucket		
Weighting g	H@10	N@10	H@10	N@10	Head	Torso	Tail
Sqrt $1/\sqrt{f+1}$	**0.7545**	**0.5347**	0.0774	0.0410	0.1158	**0.0118**	0.0016
Linear $1/(f+1)$	0.7543	0.5343	0.0770	0.0409	0.1155	0.0107	0.0021
Binary (selected)	0.7539	0.5339	**0.0775**	**0.0412**	**0.1160**	0.0115	**0.0022**

D Alignment weight λ sweep

Table 9 reports the proposed method on Video Games as λ varies, holding the weight function at the default log parameterization. Full-rank performance is stable across $\lambda \in [0.05, 0.5]$; $\lambda = 1.0$ degrades both protocols, and $\lambda = 0.01$ leaves alignment too weak. The selected $\lambda = 0.5$ achieves the highest sampled@100 score while remaining within noise of the full-rank peak at $\lambda = 0.1$.

Table 9: Alignment-weight sweep on Video Games with log frequency weighting, 3 seeds.

	Sampled@100		Full-rank	
λ	H@10	N@10	H@10	N@10
0.01	0.7526	0.5307	0.0728	0.0383
0.05	0.7530	0.5311	0.0736	0.0389
0.1	0.7538	0.5319	**0.0737**	**0.0389**
0.5 (selected)	**0.7610**	**0.5361**	0.0735	0.0388
1.0	0.7547	0.5276	0.0691	0.0365